

DAT12-2

Programming in Python

[Module page](#)

Version of 13 September 2026

About this textbook

This year, for the first time, every Bachelor course at Albert School has its own textbook, thanks to your teachers and AI.

- The textbook is a help for working on the course at home, after the class.
- The course as taught in class remains the reference, and prevails over the textbook wherever the two differ.
- This is a first version. Help us improve it by sharing your feedback with your teacher.

Contents

Preface	5
1. The development environment	6
1.1. Installing Python and an editor	6
1.2. The terminal and your first script	6
1.3. Packages and pip	7
1.4. Virtual environments	7
2. Core data types	9
2.1. Numbers: <code>int</code> and <code>float</code>	9
2.2. Text: <code>str</code>	9
2.3. The booleans: <code>bool</code>	10
2.4. Collections: <code>list</code> , <code>tuple</code> , <code>dict</code>	11
3. Control flow	14
3.1. Conditionals: <code>if</code> / <code>elif</code> / <code>else</code>	14
3.2. Loops: <code>for</code> and <code>while</code>	15
3.3. Skipping and stopping: <code>continue</code> and <code>break</code>	15
3.4. Getting the position with <code>enumerate</code>	16
3.5. Iterating over a dictionary	16
4. Functions	17
4.1. Defining and calling	17
4.2. Typed parameters and docstrings	17
4.3. Scope	17
5. File input and output	19
5.1. Reading and writing with context managers	19
5.2. Where <code>open</code> looks for a file	19
5.3. Handling errors with <code>try</code> / <code>except</code>	20
5.4. Raising an exception yourself	20
6. Debugging	22
6.1. Read the error message	22
6.2. Print and reason about state	22
7. Third-party libraries	24
7.1. Installing and importing	24
7.2. Reading documentation	24
8. Virtual environments, revisited	25
8.1. What an environment isolates	25
8.2. Reproducing from <code>requirements.txt</code>	25
9. Data processing: from CSV to CSV	27
9.1. The shape of a CSV file	27
9.2. A complete processing program	28
10. Objects: using what libraries give you	30
10.1. Attributes and methods	30
10.2. Finding out what an object offers	31
10.3. The object you are about to meet	31
11. A first encounter with APIs	33
11.1. HTTP GET and the <code>requests</code> library	33
11.2. Parsing a JSON response	33

Glossary 35

Preface

This is your first programming course, and it assumes nothing. You do not need to have written a line of code before. What you do need is a willingness to be precise: a computer does exactly what you tell it, no more and no less, and most of learning to program is learning to say what you mean without any ambiguity.

Python is our instrument. It is one of the most widely used languages in the world, and the main language of data and artificial intelligence — the field this course belongs to. We chose it because it reads almost like English, because it hides machinery you do not yet need to see, and because nearly every tool you will meet later in your studies is controlled with Python.

The course has one destination, and every chapter is a step toward it: by the end you will write a Python program that reads data from a file, transforms it, and writes a new file — and you will be able to run it again tomorrow, on another computer, and get the same answer. That last property, **reproducibility**, is what separates a program you run once and discard from a real tool. We build toward it in order: first the environment, then the language, then files, then the libraries and ideas that let your programs reach beyond your own computer.

You cannot learn programming only by reading it, any more than you can learn to swim by reading about water. So read this book with your hands on the keyboard. Type every example, change it, run it, break it on purpose, and read the message Python prints. Every chapter ends with **exercises** do them before moving on, and check your work against the answers. The error messages that beginners learn to fear are, in truth, the most patient teacher you will have — this book will teach you to read them.

1. The development environment

Before writing Python you must be able to run it. A program is just text saved in a file; what turns that text into behaviour is the **Python interpreter**, a program already installed (or installable) on your computer that reads your whole file and then carries out its instructions from top to bottom.

1.1. Installing Python and an editor

You need two things: the Python interpreter, and a good editor to write code in.

The interpreter. Download Python from the official site, python.org, and run the installer for your operating system. On Windows, tick the box labelled *Add Python to PATH* during installation — this is what lets you type `python` in a terminal later. To confirm the install worked, open a terminal (see below) and type:

```
python --version
```

It should print a version number beginning with 3, such as `Python 3.12.2`.

The editor. You could write Python in any text editor, but a code editor helps because it understands the language. We use **Visual Studio Code** (VS Code), a free editor from code.visualstudio.com. It colours your code so its structure is visible at a glance, underlines mistakes before you run anything, and includes a built-in terminal so you never leave the window to run your program. After installing it, install its official **Python extension**: open the Extensions panel (the icon showing four squares in the left bar), search for “Python”, and click *Install* on the one published by Microsoft. That extension brings the language-aware features to life.

1.2. The terminal and your first script

A **terminal** (also called a command line) is a window where you type commands as text instead of clicking buttons. In VS Code, open one with the menu *Terminal* → *New Terminal* it opens at the bottom of the window, already pointing at your project folder.

Let us run a program. In VS Code, use *File* → *Open Folder* to open a folder for this course, then create a new file in it called `hello.py` containing one line:

```
print("Hello, world!")
```

The `print` function displays whatever you put in its parentheses. Now, in the terminal, type:

```
python hello.py
```

The terminal prints `Hello, world!`. That is the whole loop you will repeat thousands of times: edit a file, run it from the terminal, read the output. The word `python` names the interpreter; `hello.py` is the **argument** you hand to it — the file to execute.

Remark. On some computers the interpreter is called `python3` rather than `python`, because an older Python 2 still exists under the plain name `python`. If `python` reports a version starting with 2, use `python3` in every command in this book.

1.3. Packages and pip

Real programs use code that other people have written and shared, packaged into units called **packages**. You install a package with `pip`, the tool that comes with Python for exactly this:

```
pip install requests
```

This downloads the `requests` package (which we use in the last chapter) and makes it available to import in your programs. But where does that package go, and what happens when two projects need two different versions of it? The answer is a **virtual environment**, which we set up next and then return to in depth in Chapter 8.

1.4. Virtual environments

If `pip` installed every package into one shared location, two projects that needed two different versions of the same package would collide. A **virtual environment** avoids this: it is a private, project-local folder holding its own copy of the interpreter and its own packages. You create one per project, right after you create the project folder:

```
python -m venv .venv      # create an environment in a folder named .venv
source .venv/bin/activate # activate it (macOS / Linux)
.venv\Scripts\activate    # activate it (Windows)
```

Once activated, your terminal prompt usually shows `(.venv)` at the start of the line, and any `pip install` now writes into that folder only. For now, simply learn the habit: **every project gets its own environment**. Chapter 8 explains why this matters.

Pitfall. Forgetting to activate the environment is the most common first-week confusion. If `import requests` fails with `ModuleNotFoundError` even though you just installed `requests`, check that `(.venv)` appears in your prompt — you almost certainly installed the package into one environment and ran your program in another. Activate the environment, then install again.

Exercises

- 1.1. Install Python and VS Code on your computer. In a terminal, run `python --version` and write down the version number it prints.
- 1.2. Create a folder for this course, open it in VS Code, and create a file `greeting.py` that prints a greeting containing your own name. Run it from the terminal and confirm the greeting appears.
- 1.3. Inside your course folder, create and activate a virtual environment named `.venv`. Confirm that `(.venv)` appears in your prompt, then install the `requests` package with `pip`.

1.4. With the environment still active, run `pip install cowsay`, then create a file `moo.py` containing the two lines `import cowsay` and `cowsay.cow("Hello")`. Run it. Then deactivate the environment with `deactivate`, run the file again, and read the error you get. Explain in one sentence why it now fails.

Answers. (1)–(3) have no single answer; you have succeeded when each command runs without error and prints what is described. (4) After `deactivate`, you are no longer in the environment where `cowsay` was installed, so Python cannot find the package and raises `ModuleNotFoundError`.

2. Core data types

Every value in Python has a **type**, and the type determines what you can do with the value: you can divide numbers but not words, you can capitalise words but not numbers. Getting the type right is the first thing to get right.

2.1. Numbers: `int` and `float`

There are two kinds of number. An **int** is a whole number — 0, 42, -7 — with no limit on its size. A **float** is a number with a decimal point — 3.14, -0.5, 2.0 — stored in a fixed amount of memory, which has a consequence we will flag shortly.

```
apples = 12           # int
price  = 2.50         # float
total  = apples * price # 30.0 -> a float, because price is a float
```

The = sign here is **assignment**: it stores the value on the right in the **variable** named on the left, so that the name stands for the value from then on. Notice that mixing an int and a float gives a float. Division always gives a float, even when it divides evenly:

```
10 / 2    # 5.0 (float) -- "true" division
10 // 3   # 3   (int)  -- "floor" division, drops the remainder
10 % 3    # 1   (int)  -- "modulo", the remainder
```

Pitfall. Floats cannot represent every decimal exactly. Typing `0.1 + 0.2` gives `0.30000000000000004`, not `0.3`. This is not a Python bug; it is how fractional numbers are stored in the binary system every computer uses. The practical rule: **never test two floats for exact equality**. Instead of `if x == 0.3`, ask whether they are close enough, for example `if abs(x - 0.3) < 1e-9` (where `abs` is the absolute value and `1e-9` means 10^{-9} , a tiny tolerance).

2.2. Text: `str`

A **string** is a sequence of characters, written between quotation marks — single or double, your choice, as long as the opening and closing marks match:

```
name = "Ada"
greeting = "Hello, " + name + "!" # 'Hello, Ada!' -- "+" joins strings
```

Because a string is an ordered sequence, you can reach into it by **position** and take a **slice** — a run of consecutive characters — exactly as you will do with lists in the next section:

```
word = "Python"
word[0]    # 'P'    -- indexing, counting from 0
word[-1]   # 'n'    -- and from the end
word[0:3]  # 'Pyt'  -- a slice: positions 0, 1, 2 (the end is excluded)
```

Strings come with many useful built-in operations, called **methods**, which you reach with a dot after the string:

```
" Ada ".strip()      # 'Ada'          -- remove surrounding spaces
"ada".upper()        # 'ADA'
"a,b,c".split(",")   # ['a', 'b', 'c'] -- a list (see below)
```

The cleanest way to build a string out of values is an **f-string**: a string written with the letter f just before the opening quote, where anything inside {} is replaced by its value.

```
apples = 12
print(f"I have {apples} apples.")    # I have 12 apples.
print(f"Each costs {2.5:.2f} euros.") # Each costs 2.50 euros.
```

The `:.2f` after the value is a **format instruction**: it means “show this number as a float with 2 digits after the decimal point”.

2.3. The booleans: bool

A **bool** is one of exactly two values, True or False. Booleans are the answers to yes/no questions, and they are what conditionals (next chapter) act on:

```
12 > 10      # True
"a" == "b"   # False
3 == 3.0     # True  -- equal in value, even across int and float
```

Here `==` (two equals signs) **compares** two values and gives a bool. Do not confuse it with a single `=`, which **assigns**. This distinction causes many early errors.

Most real questions have more than one part — “is the visitor an adult *and* does she hold a ticket?” — so booleans are built to combine. The three **logical operators** `and`, `or`, and `not` make a compound answer out of simpler ones:

```
age = 20
has_ticket = True
age >= 18 and has_ticket  # True  -- True only when BOTH sides are true
age < 18 or has_ticket    # True  -- True when AT LEAST ONE side is true
not has_ticket            # False -- flips a bool to its opposite
```

`and` is True only when both of its sides are true; `or` is True when at least one side is true; `not` reverses the single bool that follows it. Their complete behaviour fits in one small table you can always fall back on (Table 1).

a	b	a and b	a or b	not a
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Table 1: The truth table of and, or, and not. Each row is one choice of a and b: and needs both true, or needs at least one true, and not a depends on a alone (it ignores b).

Pitfall. Each side of and/or must be a **complete** condition. To test whether x equals one of two values, you cannot write `x == 1 or 2`: Python reads it as `(x == 1) or (2)`, and since any non-zero number counts as true, the whole test is always true. Spell both comparisons out in full: `x == 1 or x == 2`.

2.4. Collections: list, tuple, dict

Three types hold **many** values at once, and choosing between them is one of the recurring judgement calls of Python.

A **list** is an ordered, changeable sequence, written with square brackets. You reach an element by its **index**, counting from zero. You may also count from the end using negative indices, where -1 is the last element (Figure 1):

```
scores = [90, 85, 72]
scores[0]      # 90    -- first element
scores[-1]     # 72    -- last element
scores.append(64) # scores is now [90, 85, 72, 64]
scores[1] = 88  # change in place -> [90, 88, 72, 64]
scores[1:3]     # [88, 72] -- a slice, just like a string
len(scores)     # 4     -- how many elements
```

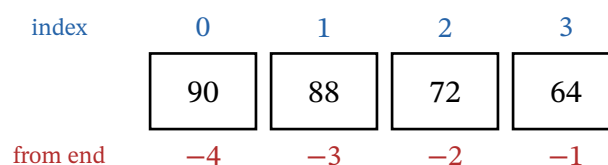


Figure 1: Indexing the list [90, 88, 72, 64]. The top number is the usual index, counting from 0; the bottom number is the negative index, counting back from the end. So `scores[1]` and `scores[-3]` name the same element, 88.

A **tuple** is an ordered sequence that **cannot** change once created, written with parentheses. Use it for a fixed group of values that belong together — a coordinate, a colour written as three numbers — where changing one without the others would be a mistake:

```
point = (3, 4)
point[0]      # 3
point[0] = 9   # TypeError: a tuple cannot be changed
```

A tuple can be **unpacked** into several variables at once, one per element — a small convenience you will use constantly:

```
point = (3, 4)
x, y = point      # x is now 3, y is now 4
```

A **dict** (dictionary) stores **key** → **value** pairs and looks things up by key rather than by position. It is the right type whenever your data is “labelled”:

```
student = {"name": "Ada", "age": 19}
student["name"]      # 'Ada'      -- look up by key
student["age"] = 20   # change a value
student["city"] = "Paris" # add a new pair
```

Type	Ordered?	Changeable?	Use it for
list	yes	yes	a sequence you grow or edit
tuple	yes	no	a fixed group that belongs together
dict	yes (insertion order)	yes	data looked up by a label
str	yes	no	text

Table 2: Choosing a collection type. “Changeable” is the word programmers shorten to *mutable* “unchangeable” is *immutable*.

Remark. You can ask any value its type with `type(x)`, and convert between types with the type’s name: `int("42")` turns the string "42" into the number 42, and `str(42)` does the reverse. Reading a number from a file always gives you a string first — converting it is a step beginners routinely forget.

Exercises

- 2.1. Predict the value and type of each of `7 / 2`, `7 // 2`, and `7 % 2`. Then check your predictions by printing them.
- 2.2. Create a variable name holding your first name and a variable age holding your age as an int. Use an f-string to print exactly: My name is <name> and next year I will be <age+1>.
- 2.3. Given `temperatures = [18, 21, 19, 23, 20]`, write expressions that give (a) the first temperature, (b) the last temperature using a negative index, and (c) the number of temperatures. Then append 22 to the list and change the second element to 25.
- 2.4. A dict `book = {"title": "Python", "pages": 300}` describes a book. Add a key "author" with your name as its value, change "pages" to 320, and print the title using an f-string.
- 2.5. You read the two strings "90" and "72" from a file. Explain why `"90" + "72"` gives "9072", and write the expression that instead gives the number 162.
- 2.6. For each item below, name the most fitting core type and say in one short phrase why it beats the others: (a) the three numbers of an RGB colour, which must always travel together; (b) a phone book that maps each name to a number; (c) a to-do list you keep adding items to; (d) whether a user is currently logged in.
- 2.7. Let `a = True` and `b = False`. Predict the value of `a and b`, `a or b`, `not a`, and `a and not b`, then check your predictions by printing them.

Answers. (1) 3.5 float, 3 int, 1 int. (2) e.g. `print(f"My name is {name} and next year I will be {age + 1}.")`. (3) `temperatures[0]`, `temperatures[-1]`, `len(temperatures)`, then `temperatures.append(22)` and `temperatures[1] = 25`. (4) `book["author"] = "..."`, `book["pages"] = 320`, `print(f"{book['title']}")`. (5) `+` on two strings joins them, so it makes the text "9072"; convert first: `int("90") + int("72")`. (6) (a) a tuple — a fixed group that belongs together and should not change; (b) a dict — data looked up by a label (the name) rather than by position; (c) a list — an ordered sequence you grow; (d) a bool — a single yes/no value. (7) `a and b` is False (both must be true), `a or b` is True (one is enough), `not a` is False, and `a and not b` is True (both `a` and `not b` are true).

3. Control flow

So far our programs run straight down, from top to bottom. **Control flow** lets a program choose what to do and repeat work — the difference between a fixed list of instructions and an actual algorithm.

3.1. Conditionals: `if` / `elif` / `else`

A conditional runs a block of code only when a condition is `True`. Consider turning a numeric score into a letter grade:

```
score = 78
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"
print(grade)          # C
```

Python checks each condition in order and runs the **first** block whose condition is true, skipping the rest. `elif` (short for “else if”) chains further alternatives; the final `else` catches everything that no condition matched.

Pitfall. Indentation is not decoration in Python — it is the syntax. The indented lines are the ones that “belong to” the `if`; if you get the indentation wrong, the meaning changes or the program refuses to run. Use four spaces per level and be consistent.

Pitfall. Do not confuse `=` with `==`: a single `=` **assigns** a value, while `==` **compares** two values. Writing `if score = 90` is an error; you mean `if score == 90`. This is one of the most common early mistakes.

A condition can test several things at once with the logical operators from Section 2.3. Suppose only an adult holding a ticket may enter:

```
if age >= 18 and has_ticket:
    print("Welcome")
else:
    print("Entry refused")
```

Reach for `and` and `or` rather than nesting one `if` inside another: the compound condition says the same thing in a single readable line.

3.2. Loops: for and while

A **for loop** repeats once for each item in a collection. This is the loop you will use most often, because most work is “do the same thing to every item”:

```
scores = [90, 85, 72]
total = 0
for s in scores:
    total = total + s
print(total)          # 247
```

To repeat a fixed number of times, loop over `range(n)`, which produces the numbers 0, 1, ..., n-1:

```
for i in range(3):
    print(i)          # prints 0, then 1, then 2
```

A **while loop** repeats as long as a condition stays true. Use it when you do not know in advance how many repetitions you will need:

```
balance = 100
years = 0
while balance < 200:
    balance = balance * 1.05    # 5% growth per year
    years = years + 1
print(years)                  # 15
```

Pitfall. A while loop whose condition never becomes false runs forever — an **infinite loop**. Make sure something inside the loop changes a value that the condition depends on. If a program seems stuck, press `Ctrl+C` in the terminal to stop it, then look at the variable your condition tests and check that the loop body really moves it toward stopping.

3.3. Skipping and stopping: continue and break

Two keywords give a loop finer control. `continue` skips the rest of the current turn and moves straight to the next item; `break` stops the loop entirely.

```
for n in [4, -1, 9, -3, 16]:
    if n < 0:
        continue          # ignore negatives, go to the next number
    print(n)              # prints 4, 9, 16

for n in [4, 8, 15, 16, 23]:
    if n > 10:
        break             # stop at the first number over 10
    print(n)              # prints 4, 8
```

3.4. Getting the position with enumerate

Sometimes you need both each item **and** its position. `enumerate` gives you both as a pair, which you unpack (as you did with tuples) into two variables:

```
names = ["Ada", "Bo", "Cy"]
for i, name in enumerate(names):
    print(f"{i}: {name}")      # 0: Ada, then 1: Bo, then 2: Cy
```

3.5. Iterating over a dictionary

Dictionaries can be looped over too. Looping over a dict directly gives its keys; its `.items()` method gives each key and value together as a pair, which you unpack — a pattern you will use constantly when processing data:

```
prices = {"apple": 2.5, "pear": 3.0}
for fruit, price in prices.items():
    print(f"{fruit} costs {price} euros")
```

Exercises

- 3.1. Write a program that, for every number from 1 to 20, prints "even" or "odd" next to the number. (Hint: use `range` and the `%` operator.)
- 3.2. Given `scores = [55, 90, 40, 70, 88]`, use a `for` loop to count how many scores are at least 60, and print the count.
- 3.3. Given `ages = [15, 18, 21, 17, 40]`, loop over the list and print only the ages that are at least 18 *and* at most 30, using an `and` condition.
- 3.4. Using a `while` loop, find the smallest number of whole years it takes an investment of 1000 to exceed 2000 while growing 7% each year. Print that number.
- 3.5. Loop over `words = ["ok", "", "hi", "", "go"]` and print only the non-empty words, using `continue` to skip the empty ones.
- 3.6. Given the dict `ages = {"Ada": 19, "Bo": 22, "Cy": 20}`, print one line per person in the form `Name is Age years old`, then print the average age.

Answers. (1) Use `for n in range(1, 21):` and test `if n % 2 == 0:`. (2) count is 3 (the scores 90, 70, 88). (3) test `if age >= 18 and age <= 30:`; it prints 18 and 21. (4) 11 years. (5) prints `ok, hi, go`. (6) average is 20.33...; compute it by summing the values and dividing by `len(ages)`.

4. Functions

As soon as you find yourself writing the same lines twice, you want a **function**: a named, reusable block of code that takes inputs and (usually) returns an output. Functions are how programs are kept small enough to understand — each one is a box you can reason about without looking inside the others.

4.1. Defining and calling

Suppose we keep converting temperatures from Celsius to Fahrenheit. We write the rule once:

```
def to_fahrenheit(celsius):
    return celsius * 9 / 5 + 32

print(to_fahrenheit(100)) # 212.0
print(to_fahrenheit(0))  # 32.0
```

`def` introduces a function; `to_fahrenheit` is its name; `celsius` is a *parameter*, a placeholder filled in when you **call** the function. `return` hands a value back to whoever called it. A function that has no `return` hands back the special value `None`, which stands for “no value”.

4.2. Typed parameters and docstrings

Python lets you annotate what types a function expects and returns. These **type hints** do not change how the code runs, but they document your intent and let your editor catch mistakes:

```
def to_fahrenheit(celsius: float) -> float:
    """Convert a temperature from Celsius to Fahrenheit."""
    return celsius * 9 / 5 + 32
```

The `: float` marks the parameter’s expected type, and `-> float` marks the type of the returned value. The triple-quoted string just under the `def` line is a **docstring**: a sentence saying what the function does. It is not only a note to yourself — tools display it as the function’s official help, and running `help(to_fahrenheit)` prints it. Write one for every function.

4.3. Scope

A variable created inside a function exists only inside it — its **scope** is local to that one call. This is a feature, not a limitation: it means a function’s inner workings cannot accidentally overwrite a variable somewhere else.

```
def square(x):
    result = x * x      # 'result' lives only inside square
    return result

print(square(5))       # 25
print(result)          # NameError: 'result' is not defined out here
```

Pitfall. The opposite direction is a classic trap. A function **can read** a variable defined outside it, so a function that forgets to take a value as a parameter may appear to work by silently using an outside variable — until that outside variable changes and the function’s behaviour changes with it, mysteriously. Pass everything a function needs as parameters; do not rely on values reaching in from the surrounding code.

Exercises

- 4.1. Write a function `area_of_rectangle(width, height)` that returns the area. Add type hints and a docstring. Call it with a few values and print the results.
- 4.2. Write a function `is_passing(score)` that returns `True` if the score is at least 60 and `False` otherwise. Use it inside a loop over a list of scores to print each score and whether it passed.
- 4.3. Write a function `initials(first, last)` that returns a person’s initials as a two-letter string, for example "AL" for "Ada", "Lovelace".
- 4.4. Explain, in one sentence, why the program below prints `None` instead of the discounted price, and fix it so the function returns that price.

```
def apply_discount(price):
    discounted = price * 0.9
    print(apply_discount(100))
```

Answers. (1) the full definition carries both a type-hinted signature and a docstring:

```
def area_of_rectangle(width: float, height: float) -> float:
    """Return the area of a rectangle."""
    return width * height
```

(2) return `score >= 60`. (3) e.g. return `first[0].upper() + last[0].upper()`. (4) The function has no return, so it hands back `None`, which is what `print` shows. Fix: add `return discounted` as the function’s last line.

5. File input and output

A program that only prints to the screen forgets everything when it ends. To do real work it must read files and write files — to **persist** data.

5.1. Reading and writing with context managers

Opening a file gives you a connection to it that must eventually be closed, or you risk losing data and holding resources you no longer need. The clean way is the `with` statement — a **context manager** that closes the file automatically when the block ends, even if something goes wrong inside it:

```
# Writing
with open("notes.txt", "w") as f:    # "w" = write (replaces the file)
    f.write("first line\n")          # \n is a newline character
    f.write("second line\n")

# Reading
with open("notes.txt", "r") as f:    # "r" = read (this is the default)
    content = f.read()               # the whole file as one string
print(content)
```

The variable `f` is the open file, valid only inside the `with` block. To read a file line by line — the usual case for data — loop over it directly:

```
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())          # .strip() removes the trailing newline
```

Pitfall. The mode matters. Opening with `"w"` **erases** the file before writing — an easy way to destroy data you meant to keep. Use `"a"` (append) to add to the end of an existing file, and `"r"` to read without any risk of changing it.

5.2. Where `open` looks for a file

`open("notes.txt")` names the file but not the folder, so where does Python look? A bare filename is resolved relative to the **current working directory** — the folder your terminal is sitting in **when you launch the program**, which is not necessarily the folder the `.py` file lives in. When you run `python script.py` from your project folder and the data file sits in that same folder, the two agree and `open` finds it. Run the very same program from a different folder and `open("notes.txt")` now looks in **that** folder instead, and fails even though the file is plainly there in your editor. This is why the VS Code terminal, which opens already pointing at your project folder (Chapter 1), keeps the common case simple: keep a program and the files it reads together, and run it from their folder. To place a file in a subfolder, name the path: `open("data/notes.txt")`.

Pitfall. When `open` raises `FileNotFoundError` but you can see the file right there in the editor, the file almost always exists — Python simply looked somewhere else. It searches the current working directory, so a bare name only finds the file when you run the program from the folder that holds it. Check where your terminal is (run `pwd` on macOS/Linux, or `cd` on Windows), then either run the program from that folder or hand open the full path.

5.3. Handling errors with `try / except`

What if the file is not there? Or a line you expected to be a number turns out to be the word “unknown”? An **exception** is Python’s way of signalling that something went wrong; if you do nothing about it, it stops the program. The `try / except` block lets you catch the problem and respond instead of crashing:

```
try:
    with open("data.csv", "r") as f:
        content = f.read()
except FileNotFoundError:
    print("No data file found; starting empty.")
    content = ""
```

Python **tries** the first block; if the named exception occurs, it runs the `except` block instead. You can handle different problems differently by naming the exception you expect:

```
try:
    value = int(line)
except ValueError:
    print(f"Skipping line that is not a number: {line!r}")
```

The `!r` in that message earns a word, because we lean on it again when debugging. It asks for the value’s **repr** — its programmer-facing form, with the quotation marks kept and normally-invisible characters made visible. Where `{line}` prints `unknown`, `{line!r}` prints `'unknown\n'`, exposing the trailing newline that reading a file leaves on the end of every line. That invisible newline is a classic source of confusion: `"yes\n" == "yes"` is `False`, which is why you `.strip()` a line before comparing it. When a value misbehaves and you cannot see why, printing it with `!r` is the quickest way to see what it **really** holds.

5.4. Raising an exception yourself

You can also **raise** an exception on purpose, to signal that something your own code was asked to do makes no sense. Use the `raise` keyword with the kind of error and a message:

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("insufficient funds")
    return balance - amount
```

If someone calls `withdraw(100, 150)`, Python stops and reports a `ValueError` with your message — unless the caller wrapped the call in a `try / except` to handle it. Raising and catching are two halves of the same idea: one signals a problem, the other responds to it.

Pitfall. Resist the urge to write a bare `except :` that catches **every** error. It hides the very bugs you most need to see — a typo in a variable name also raises an exception, and a blanket `except` will quietly bury it. Catch the specific exception you expect, and let the unexpected ones surface.

Exercises

- 5.1.** Write a program that writes the numbers 1 to 5, one per line, into a file `numbers.txt`, then reads the file back and prints each line.
- 5.2.** Modify the previous program to open the file a second time in append mode ("`a`") and add the numbers 6 and 7. Read and print the file to confirm all seven numbers are there.
- 5.3.** Write a program that tries to open a file `missing.txt` for reading and, if the file does not exist, prints `File not found` instead of crashing.
- 5.4.** Write a function `safe_int(text)` that returns `int(text)` if it can, and returns `None` (without crashing) if `text` is not a valid number. Test it on `"42"` and on `"hello"`.
- 5.5.** Write a function `sqrt_or_error(x)` that returns the square root of `x` (use `x ** 0.5`) but raises a `ValueError` with a clear message if `x` is negative.

Answers. (1) write with a loop, then read it back: open `"numbers.txt"` in mode `"w"` and inside the block for `n` in `range(1, 6): f.write(f"{n}\n")`; then reopen in `"r"` and for `line` in `f: print(line.strip())`. It prints 1 to 5, one per line. (2) reopen the same file in mode `"a"` and write 6 and 7 the same way; reading it back now prints 1 through 7, because `"a"` adds to the end instead of erasing. (3) Wrap the open in `try: ... except FileNotFoundError: print("File not`

`found")`. (4) `try: return int(text) except ValueError: return None`. (5) if `x < 0: raise ValueError("cannot take the square root of a negative number")`, then return `x ** 0.5`.

6. Debugging

Your programs will not work the first time. Nobody's do. **Debugging** — finding and fixing the gap between what you told the computer and what you meant — is not a sign of failure; it is the main activity of programming. The good news is that it is a learnable, systematic skill, not a talent you are born with.

6.1. Read the error message

When Python hits an error it prints a **traceback**: a report that ends with the error type and message, and points at the line where it happened. Beginners skim past it in a panic; experienced programmers read it carefully, from the bottom up. The last line is usually the most informative:

```
Traceback (most recent call last):
  File "crash.py", line 7, in <module>
    total = total + scores[5]
                        ~~~~~^^^
IndexError: list index out of range
```

This tells you **what** (an `IndexError`), **where** (line 7), and **why** (a list index out of range — you asked for `scores[5]` in a list that does not have six items). Three of the four questions are answered before you have even started to think. Learning the common error types — `NameError`, `TypeError`, `ValueError`, `IndexError`, `KeyError`, `FileNotFoundError` — turns most tracebacks into an instant diagnosis (Table 3).

Error type	It usually means
<code>NameError</code>	you used a variable or function name that does not exist
<code>TypeError</code>	an operation was applied to the wrong type of value
<code>ValueError</code>	the type was right but the value was not, e.g. <code>int("hi")</code>
<code>IndexError</code>	a list index points past the end of the list
<code>KeyError</code>	a dict has no such key
<code>FileNotFoundError</code>	Python found no file where it looked — often the right name in the wrong folder

Table 3: The errors a beginner meets most often, and what each one is telling you.

6.2. Print and reason about state

When the message alone is not enough, the oldest tool still works: print the values your code depends on, just before the line that misbehaves, and check whether they are what you assumed. Suppose `lines` is a list of lines you read from a file and `int(line)` keeps failing:

```
for i, line in enumerate(lines):
    print(f"line {i}: {line!r}") # what is REALLY in each line?
    value = int(line)
```

Programming is reasoning about **program state** — the current values of all your variables. A bug is almost always a place where the real state differs from the state you imagined. Printing makes the real state visible; the moment you see it, the bug usually explains itself.

Remark. Recall `!r` from Chapter 5: it prints a value's **repr**, so a line read from a file shows as `'42\n'`, not `42`, with its trailing newline and any surrounding spaces laid bare. When a line refuses to convert or a comparison gives an impossible answer, `print(f"{line!r}")` is the fastest way to see the hidden characters behind it.

Remark. VS Code, the editor you set up in Chapter 1, offers a third tool: a built-in **debugger**. Click just left of a line number to set a **breakpoint**, then start the file from the *Run and Debug* panel; the program runs until it reaches that line and pauses, letting you inspect the value of every variable at that exact moment — the same program state a print reveals, but shown without editing your code. A print stays the quickest first reflex and works anywhere your code runs; when a bug hides in a tangle of state, the debugger shows more of it at once.

Exercises

6.1. The program `print(total)` raises a `NameError`. What does that tell you, and what is the most likely cause?

6.2. Read this traceback and answer, in your own words, what went wrong and on which line:

```
Traceback (most recent call last):
  File "grades.py", line 3, in <module>
    average = total / count
ZeroDivisionError: division by zero
```

6.3. The code below is meant to print 6 but instead stops with an error. Add a print to inspect the state, find the bug, and fix it.

```
numbers = ["1", "2", "3"]
total = 0
for n in numbers:
    total = total + n
print(total)
```

6.4. Match each error to its cause: `KeyError`, `IndexError`, `TypeError` — for the expressions `[1, 2][5]`, `{"a": 1}["b"]`, and `"3" + 3`.

Answers. (1) A name was used before it was given a value — likely a typo in the variable name or a line that never ran. (2) On line 3, `count` was 0, so the division by zero failed. (3) The list holds strings, so `total + n` tries to add a string to a number and raises a `TypeError`; convert with `int(n)`, i.e. `total = total + int(n)`. (4) `[1,2][5] → IndexError`, `{"a":1}["b"] → KeyError`, `"3" + 3 → TypeError`.

7. Third-party libraries

Python’s real power is its **ecosystem**: hundreds of thousands of packages written by others, each one a problem already solved. Learning to find, install, and use them is as important as the language itself.

7.1. Installing and importing

We met `pip` in the first chapter. Inside an activated virtual environment, install a package once:

```
pip install requests
```

Then **import** it in your program to make its contents available. There are three common forms of import:

```
import requests                # then use it as requests.get(...)
from datetime import date      # bring one name out of a package
import statistics as stats     # import under a shorter alias: stats.mean(...)
```

The first form imports the whole package under its own name. The second brings a single name out of a package so you can use it directly. The third imports a package under a shorter **alias**, handy for names you will type often. (`datetime` and `statistics` are part of Python itself, so they need no `pip` install; `requests` is third-party, so it does.)

7.2. Reading documentation

You cannot — and need not — memorise a library. The real skill is reading its **documentation**: the official description of what each function expects and returns. When you meet a new library, find its documentation, look at the first example, copy it, run it, then adapt it to your need. Every library worth using has a “Quickstart” page written for exactly this moment. Learning to read documentation well is the single habit that most separates a self-sufficient programmer from a stuck one.

Exercises

7.1. The `statistics` module is part of Python. Import it and use it to print the mean and the median of the list `[4, 8, 15, 16, 23, 42]`. (Find the function names by searching for “python statistics module”.)

7.2. Using `from datetime import date`, print today’s date. Search the documentation for how to get the current date.

7.3. Install the `requests` package in your virtual environment (if you have not already). Confirm the install by running a program that only contains `import requests` — it should run without error.

7.4. Open the documentation for the `statistics` module and find a function that returns the **most common** value in a list. Use it on `[1, 2, 2, 3, 3, 3, 4]`.

Answers. (1) `statistics.mean(...)` gives 18, `statistics.median(...)` gives 15.5. (2) `print(date.today())`. (3) has no single answer; you have succeeded when the program containing only `import requests` runs and prints nothing, with no `ModuleNotFoundError`. (4) `statistics.mode(...)` gives 3.

8. Virtual environments, revisited

We created a virtual environment back in the first chapter; now, with packages to isolate, we can say **why** it matters.

8.1. What an environment isolates

A virtual environment isolates **which packages, at which versions** a project sees. Project A can use version 1 of a library while project B uses version 2, with no conflict, because each has its own `.venv` folder (Figure 2). It does not isolate your files or your data — only the installed packages and the interpreter used to run them.



Figure 2: Each project keeps its own `.venv` with its own package versions, so the two never collide — even when they need different versions of the same library.

8.2. Reproducing from `requirements.txt`

Here is where isolation pays off. Inside an activated environment, freeze the exact versions you are using into a file:

```
pip freeze > requirements.txt
```

The file lists every installed package and its exact version:

```
certifi==2026.7.22
charset-normalizer==3.5.1
idna==3.19
requests==2.32.5
urllib3==2.7.0
```

Anyone — a teammate, a grader, or **you** in six months on a new computer — can now recreate the same set of packages:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt    # -r = install the packages listed in this file
```

This is the heart of **reproducibility**, the property this whole course builds toward. A program together with its `requirements.txt` is a reliable recipe: install these packages and you will get the same result its author did.

Remark. One honest limit: `requirements.txt` records your **packages** and their versions, but not the version of Python itself, nor your operating system. Two computers with very different Python versions can still behave differently. So, next to your `requirements` file, also write down the Python version you used (the one `python --version` prints). Keep your program and its `requirements.txt` together — these are what you share and what you back up; you never need to keep the `.venv` folder itself, which is large, computer-specific, and can always be rebuilt from the `requirements` file.

Exercises

8.1. In an activated environment with `requests` installed, run `pip freeze > requirements.txt` and open the file. How many packages are listed, and why are there more than the one you installed?

8.2. Create a **second** folder with its own fresh virtual environment, and use the `requirements.txt` from the first to install the same packages with `pip install -r requirements.txt`. Confirm `import requests` works there.

8.3. Explain, in two sentences, why you should keep and share `requirements.txt` but not the `.venv` folder.

8.4. Your colleague sends you a program and its `requirements.txt`, but it behaves differently on your computer. Name one thing the `requirements` file does **not** capture that could explain the difference.

Answers. (1) Five in a typical install (`requests` plus `certifi`, `charset-normalizer`, `idna`, and `urllib3`): `pip` installs `requests`'s own dependencies automatically, so the file lists more packages than the one you asked for. (2) has no single answer; you have succeeded when `pip install -r`

`requirements.txt` finishes in the fresh environment and a program with only `import requests` then runs there without a `ModuleNotFoundError`. (3) `requirements.txt` is small and lets anyone rebuild the environment; `.venv` is large, specific to your computer, and fully rebuildable, so keeping it wastes space. (4) The Python interpreter version (or the operating system).

9. Data processing: from CSV to CSV

We now assemble everything into the program the whole course points at: read a data file, process it, write a new one (Figure 3). The standard format for tabular data is **CSV** — comma-separated values, a plain-text table in which each line is a row and commas separate the columns.

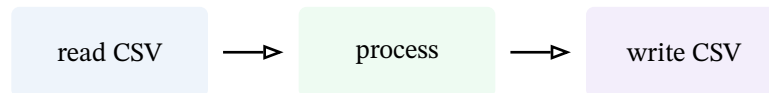


Figure 3: The shape of every data-processing program, and the destination of this course: read a file, transform what it holds, write a new file.

9.1. The shape of a CSV file

Suppose `students.csv` contains:

```
name,score
Ada,90
Bo,72
Cy,55
```

The first line is a **header** naming the columns. Python's built-in `csv` module reads such a file, and `csv.DictReader` hands each row back as a dictionary keyed by the header — exactly the labelled data a dict is made for:

```
import csv

with open("students.csv", "r", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row)
```

Run it (from the folder that holds `students.csv`) and it prints one dictionary per row:

```
{'name': 'Ada', 'score': '90'}
{'name': 'Bo', 'score': '72'}
{'name': 'Cy', 'score': '55'}
```

Pitfall. Look closely: `row["score"]` is the **string** `'90'`, not the number `90` — everything read from a CSV arrives as text. If you want to compute with it, you must convert first: `int(row["score"])`. Forgetting this gives the baffling result `"90" + "72" = "9072"` instead of `162`, because `+` on strings joins them.

9.2. A complete processing program

Here is the destination, in full: read the students, keep only those who passed, and write them to a new file with an added column. It uses data types, a loop, a conditional, type conversion, file input and output, and error handling together:

```
import csv

PASS_MARK = 60

def load_students(path: str) -> list:
    """Read a CSV of students into a list of dicts, scores as ints."""
    students = []
    with open(path, "r", newline="") as f:
        for row in csv.DictReader(f):
            try:
                row["score"] = int(row["score"])
            except ValueError:
                print(f"Skipping bad row: {row!r}")
                continue # skip this row, go to the next one
            students.append(row)
    return students

def write_passers(students: list, path: str) -> None:
    """Write students who passed, with a 'result' column, to a new CSV."""
    with open(path, "w", newline="") as f:
        writer = csv.DictWriter(f, fieldnames=["name", "score", "result"])
        writer.writeheader()
        for s in students:
            if s["score"] >= PASS_MARK:
                s["result"] = "PASS"
                writer.writerow(s)

students = load_students("students.csv")
write_passers(students, "passers.csv")
print(f"Processed {len(students)} students.")
```

Run it, and it prints its one-line summary:

```
Processed 3 students.
```

and a new file `passers.csv` appears next to `students.csv`, holding Ada and Bo with a `result` column added, and Cy — below the pass mark — dropped:

```
name,score,result
Ada,90,PASS
Bo,72,PASS
```

Notice how the work split naturally into two small, named functions — one to read, one to write — each of which you can test on its own. That structure is not decoration; it is what makes a program you can debug and trust.

Remark. The `newline=""` is not optional clutter, and it belongs on **both** the read and the write. The `csv` module manages line endings itself: on writing, leaving `newline=""` out gives an extra blank line between every row on Windows; on reading, it is what lets the module correctly handle a newline that appears **inside** a quoted field. The simple rule: pass `newline=""` whenever you hand a file to the `csv` module, reading or writing.

Exercises

- 9.1.** Create the file `students.csv` shown above. Write a program that reads it with `csv.DictReader` and prints, for each student, `Name scored Score` using an f-string (remember the score arrives as a string).
- 9.2.** Extend your program to compute and print the average score across all students. Convert each score to an int first.
- 9.3.** Write a program that reads `students.csv` and writes a new file `honours.csv` containing only the students who scored at least 85, keeping the same two columns.
- 9.4.** Add a deliberately broken row to `students.csv` (for example `Dot,abc`) and confirm the complete program above skips it without crashing. Explain which line makes that happen.
- 9.5.** Modify `write_passers` to also add a `"result"` of `"FAIL"` (instead of dropping the student) for those below the pass mark, so every student appears in the output.

Answers. (1) loop over `csv.DictReader(f)` and, for each row, `print(f'{row["name"]} scored {row["score"]}')`; it prints `Ada scored 90, Bo scored 72, Cy scored 55` (the score stays a string, which is fine here). (2) sum the converted scores and divide by the count; the average of 90, 72, 55 is 72.33... (3) read with `csv.DictReader`, and with a `csv.DictWriter(f, fieldnames=["name", "score"])` write the header and only the rows where `int(row["score"]) >= 85`; `honours.csv` then holds just Ada. (4) The `try / except ValueError` around `int(row["score"])` catches the bad conversion and continue skips the row. (5) remove the if guard around the write, and set `s["result"]` to `"PASS"` or `"FAIL"` based on the score before writing every row.

10. Objects: using what libraries give you

You have been using **objects** since the first chapter, without the name. When you wrote `"ada".upper()` or `scores.append(64)`, the value to the left of the dot was an object, and the name to the right was something that object provides. An **object** is a value that bundles **data** together with **behaviour**: the data are its *attributes*, the behaviour its *methods*. Almost every value a library hands back is an object, so using a library well is, in large part, knowing how to read the objects it returns. You will not define your own objects in this course — that comes in a later one — but you will use them constantly, and this short chapter is the lens for doing so.

10.1. Attributes and methods

Two kinds of thing hang off an object, both reached with a dot, and the whole difference between them is whether you add parentheses.

An **attribute** is a piece of **data** stored on the object. You read it with a dot and **no** parentheses — you are just looking at a value that is already there:

```
word = "Python"
# an attribute is data you read; a method is an action you run (below)
```

A **method** is a **function that belongs to the object** — a piece of behaviour. You call it with a dot **and** parentheses, because calling it does some work and usually hands back a result:

```
"ada".upper()      # 'ADA'           -- a method: () because it does something
[3, 1, 2].sort()   # sorts the list in place
"a,b,c".split(",") # ['a', 'b', 'c'] -- a method that returns a new value
```

The parentheses are the whole signal (Figure 4): reach for an attribute and you get the stored value; call a method and you get the result of running it.

```
response.status_code  —> 200
no () → an attribute (data you read)

response.json()       —> a dict
with () → a method (behaviour you run)
```

Figure 4: The dot reaches two kinds of thing, and the parentheses tell them apart: `response.status_code` reads an attribute — the stored number 200 — while `response.json()` calls a method, which runs and returns a dict.

Pitfall. Confusing the two fails loudly, which is a mercy. Add parentheses to an attribute and Python tries to **call** a value that is not a function: `response.status_code()` raises `TypeError: 'int' object is not callable`. Forget the parentheses on a method and you get the method itself instead of its result — printing `response.json` shows something like `<bound method Response.json ...>`, not your data. So when a value prints as `<bound method ...>`, you left off a pair of parentheses.

10.2. Finding out what an object offers

You do not memorise an object’s attributes and methods; you look them up. Three tools answer the questions “what is this?” and “what can it do?”:

- `type(x)` names the object’s **kind**. Later you will see `type(response)` report `<class 'requests.models.Response'>` — a `Response` object, defined by the `requests` library.
- The library’s **documentation** lists what that kind offers and is the real source of truth; you practised reading it in Chapter 7.
- `dir(x)` lists the names available on an object — a quick reminder when the documentation is not in front of you.

Between them, `type` tells you **what you are holding** and the documentation tells you **what you may do with it**.

10.3. The object you are about to meet

You have already used objects returned by libraries without dwelling on it: in the last chapter, `csv.DictReader(f)` handed you an object you looped over, and `csv.DictWriter(...)` handed you one whose `.writeheader()` and `.writerow()` **methods** did the writing. The next chapter makes the pattern explicit. It sends a request over the internet with the `requests` library, and what comes back is a single object — a **Response** — carrying everything about the server’s reply. You read part of its answer as **attributes** and run part as **methods** (Table 4):

On a response	Kind	What it gives you
<code>response.status_code</code>	attribute	the reply’s status number, e.g. 200
<code>response.ok</code>	attribute	True when the status means success
<code>response.url</code>	attribute	the address that was actually fetched
<code>response.json()</code>	method	parses the reply’s JSON into a dict or list

Table 4: A few of the members of a `requests Response` object. The attributes are data already computed for you; the method runs work when you call it. The next chapter puts them to use.

The pattern is the whole lesson: a library hands you an object, you read its attributes for data and call its methods for behaviour, and its documentation tells you which name is which. Every library you meet afterwards works this way.

Exercises

10.1. For each expression, say whether the name after the dot is an attribute or a method, and how you can tell **without running it**: `text.upper()`, `response.status_code`, `scores.append(5)`, `response.url`.

10.2. A response object has a `status_code` attribute holding 200. Explain what `response.status_code()` does and why, then write the correct expression to read the status.

10.3. You are handed an object `r` from a library you have never used, and you want to know what it is and what it offers. Name the one function that reports its kind, and the one place that authoritatively lists its methods.

10.4. A classmate writes `data = response.json` and is puzzled that `data` prints as `<bound method Response.json ...>` instead of the parsed data. What did they forget, and what is the fix?

Answers. (1) `text.upper()` and `scores.append(5)` are **methods** — they carry parentheses and perform an action; `response.status_code` and `response.url` are **attributes** — no parentheses, just stored data. (2) It tries to **call** the integer 200 as if it were a function and raises `TypeError: 'int' object is not callable`; drop the parentheses and read `response.status_code`. (3) `type(r)` reports its kind; the library's documentation authoritatively lists its methods (`dir(r)` gives a quick reminder). (4) They forgot the parentheses that **call** the method; writing `response.json()` runs it and returns the parsed data.

11. A first encounter with APIs

The last reach beyond your own computer: getting data from the internet on demand. An **API** (Application Programming Interface) is a defined way for one program to ask another for data or services. Web APIs let your program request data from a server — weather, exchange rates, public statistics — and receive a structured reply (Figure 5).

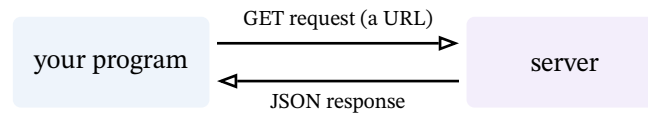


Figure 5: An HTTP GET: your program sends a request naming a URL, and the server sends back data, most often as JSON.

11.1. HTTP GET and the requests library

The web runs on **HTTP**, a protocol of requests and responses. The simplest request is a **GET**: “please give me the data at this address (URL).” The requests library makes a GET in one line:

```
import requests

response = requests.get("https://api.github.com/users/python")
print(response.status_code)  # 200 means success
```

The **status code** is the server’s verdict: 200 means OK, 404 means not found, 500 means the server itself failed. Always check it before trusting the reply. As Chapter 10 described, response is an object: status_code is one of its **attributes** — data you read, with no parentheses — and .json(), which we use next, is one of its **methods**.

11.2. Parsing a JSON response

Web APIs almost always answer in **JSON** — a text format for nested data that maps cleanly onto Python’s dicts and lists. requests turns it into Python values for you with .json():

```
import requests

try:
    response = requests.get("https://api.github.com/users/python", timeout=10)
    if response.status_code == 200:
        data = response.json()  # a dict, parsed from JSON
        print(data["name"])    # 'Python'
        print(data["public_repos"])  # an int
    else:
        print(f"Request failed with status {response.status_code}")
except requests.exceptions.RequestException as e:
    print(f"Could not reach the server: {e}")
```

Once .json() hands you a dict, you are working with an ordinary dict again: reach into it by key exactly as with the dictionary in Table 2, loop over lists of results, and pull out the fields you need.

An API call is, in the end, just one more source of the labelled data you already know how to process — and a natural partner for the CSV program of Chapter 9: fetch with `requests`, parse with `.json()`, write with `csv`.

Pitfall. Network requests fail in ways file reads do not — the server may be down, the connection may drop, the address may be wrong. So wrap a real request in `try / except` (catching `requests.exceptions.RequestException`), pass a `timeout` so your program cannot hang forever, and always check `status_code`. A program that assumes the internet always answers will crash the first time the connection drops.

Remark. Many public APIs limit how often you may call them without an account. The GitHub endpoint above, for instance, allows only about 60 unauthenticated requests per hour from one connection; call it in a tight loop and you will start receiving 403 status codes. When you experiment, call an API sparingly, and read its documentation for its rate limits before building anything on top of it.

Exercises

11.1. Run the first GET example above and print the status code. What code do you get, and what does it mean?

11.2. Fetch `https://api.github.com/users/python`, parse the JSON, and print the values of the keys `"name"` and `"public_repos"`.

11.3. Wrap your request from the previous exercise in a `try / except` that catches `requests.exceptions.RequestException`, and test it by temporarily changing the URL to an address that does not exist.

11.4. Write a program that fetches the same user, and if the request succeeds, writes a one-row CSV file `user.csv` with columns `name` and `public_repos` using the `csv` module — joining the API and CSV skills into one pipeline.

Answers. (1) You should get 200, meaning the request succeeded. (2) call `.json()` on the response to get a dict, then `print(data["name"])` and `print(data["public_repos"])`; `name` prints `Python` and `public_repos` prints the current repository count, an integer. (3) A bad URL raises a `RequestException`, which your `except` block should catch and report. (4) build a dict from the JSON, open the output with `newline=""`, and use `csv.DictWriter` with `fieldnames=["name", "public_repos"]`.

Glossary

For self-study, the key terms of this course, gathered in one place.

argument a value you hand to a function or to the interpreter when you call it.

attribute a piece of data stored on an object, read with a dot and no parentheses:
`response.status_code`.

bool a type with exactly two values, `True` and `False`.

context manager the `with` statement, which cleans up (e.g. closes a file) for you.

CSV comma-separated values, a plain-text table format.

dict a collection of key → value pairs, looked up by key.

docstring the string just under a `def` line describing what a function does.

exception Python's signal that something went wrong; caught with `try / except`.

f-string a string prefixed with `f` in which `{}` is replaced by a value.

float a number with a decimal point, stored with limited precision.

function a named, reusable block of code that takes inputs and returns an output.

index the position of an element in a sequence, counting from 0.

int a whole number of unlimited size.

interpreter the program that reads your Python file and carries out its instructions.

JSON a text format for nested data, mapping onto Python dicts and lists.

list an ordered, changeable sequence, written with square brackets.

logical operator `and`, `or`, or `not` — combines or negates booleans.

method a function that belongs to an object, called with a dot and parentheses.

mutable changeable after creation (a list is mutable; a tuple is not).

object a value that bundles data (attributes) with behaviour (methods); e.g. a string, a list, or a `requests.Response`.

package a unit of shared code you install with `pip` and bring in with `import`.

parameter a placeholder in a function definition, filled in when the function is called.

repr the programmer-facing form of a value, shown by `!r` in an f-string.

reproducibility the property that a program gives the same result elsewhere and later.

scope the region of a program where a variable exists.

status code a number in an HTTP response reporting success (200) or failure.

str a string, a sequence of characters written between quotation marks.

traceback the report Python prints when an error stops the program.

tuple an ordered sequence that cannot change after it is created.

type the kind of a value, which decides what you can do with it.

virtual environment a project-local folder holding its own packages and interpreter.